

Attention Is All You Need

<https://arxiv.org/abs/1706.03762>

▼ 1. Intro

기존 RNN/LLSTM/GRU 등 모델은 generate a sequence of hidden states h_t as a function of the previous state h_{t-1} and the input for position t .

- parallelization 배제 → 긴 시퀀스에서 치명적 왜냐면 memory constraints limit batching across examples
- Attention은 allow modeling of dependencies without regard to their distance in the input or output sequences HOWEVER such attention mechanisms are used in conjunction with recurrent network.

Transformer:

- recurrence 피하고
- instead rely on an attention mechanism to draw global dependencies between input and output
- allow more parallelization
- new SOTA in translation equality

▼ 2. Background

기존 모델의 한계 (순차적 연산)

- 기존의 Sequence transduction 모델(RNN, LSTM 등)은 입력 시퀀스를 처리할 때 이전 단계의 hidden state(h_{t-1})를 반드시 거쳐야 했음.
- 이러한 sequential 특성은 모델의 학습 효율을 떨어뜨리고 병렬 처리를 불가능하게 하여 긴 문장 처리 시 큰 제약

Convolutional 기반 모델의 한계 (거리 의존성)

- Extended Neural GPU, ByteNet, ConvS2S와 같은 모델들은 병렬 연산을 위해 CNN을 도입
- 하지만 CNN 구조에서는 입력 시퀀스 내의 먼 거리에 있는 두 지점 사이의 관계를 파악하려면 여러 층(layer)을 거쳐야 함.
- 이때 거리가 멀어질수록 필요한 연산 횟수가 증가(linear 혹은 logarithmic)하게 되어 장거리 의존성(long-range dependencies)을 학습하는 데 어려움

Transformer의 차별점 (Attention 기반)

- Transformer는 재귀(recurrence)나 합성곱(convolution)을 완전히 배제
- 대신 Attention 메커니즘을 사용하여 입력과 출력 사이의 모든 위치를 직접 연결함으로써 거리에 상관없이 단 한 번의 연산으로 의존성을 파악할 수 있게 설계
- 덕분에 parallelization 가능, 더 짧은 시간 안에 더 높은 성능(BLEU score)을 달성할 수 있었음.

▼ 3. Model Architecture

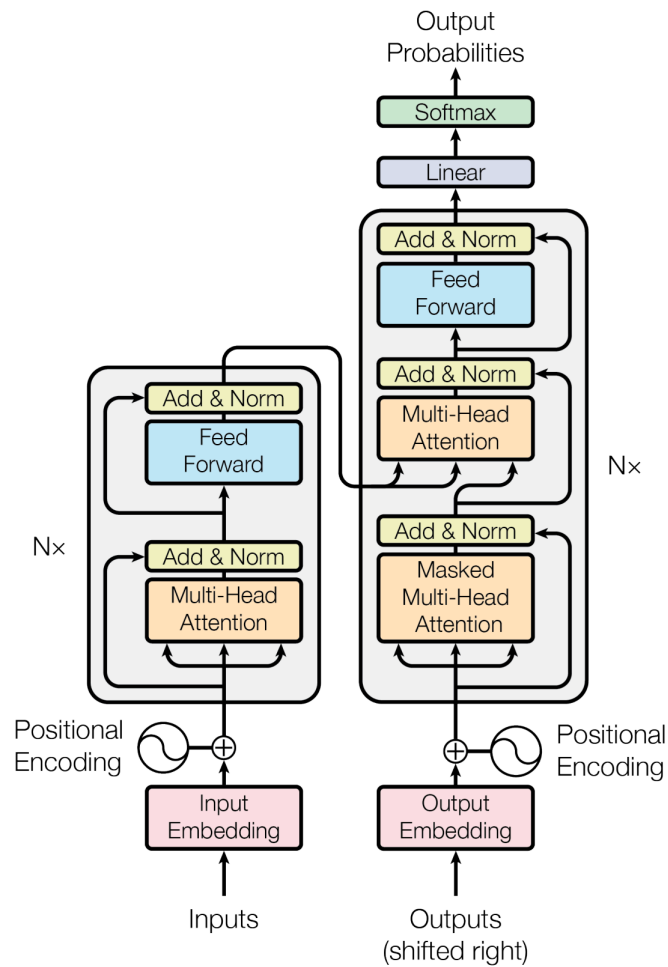


Figure 1: The Transformer - model architecture.

▼ 3.1 Encoder and Decoder Stacks

마지막 인코더 레이어의 출력이 모든 디코더 레이어에 입력된다

Encoder:

a stack of $N = 6$ identical layers. Each layer has two sub-layers.

1. The first sub-layer: a multi-head self-attention mechanism
2. the second sub-layer: a simple, positionwise fully connected feed-forward network.

employ a residual connection around each of the two sub-layers, followed by layer normalization. That is, the output of each sub-layer is $\text{LayerNorm}(x + \text{Sublayer}(x))$, where $\text{Sublayer}(x)$ is the function implemented by the sub-layer itself.

To facilitate these residual connections, all sub-layers in the model, as well as the embedding layers, produce outputs of dimension

$$d_{model} = 512$$

Decoder:

이전까지 생성된 단어들을 바탕으로 다음에 올 단어를 예측하는 역할

a stack of $N = 6$ identical layers. Each layer has two sub-layers

1. The first sub-layer: a multi-head self-attention mechanism
2. the second sub-layer: a simple, positionwise fully connected feed-forward network.
3. the third sub-layer: performs multi-head attention over the output of the encoder stack.

employ residual connections around each of the sub-layers, followed by layer normalization.

Masked Self-Attention:

디코더의 목표는 한 문장에서 순차적으로 토큰을 생성하는 것(auto-regressive)이므로, 이전에 생성한 단어들의 관계를 파악하여 앞으로 나올 토큰을 생성(예측)해야 함. 이때 인코더로부터 입력 문장 전체를 본다면 예측이 아니라 정답을 보고 베껴쓰는 게 되긴 때문에 masking이 필요함.

We also modify the scaled dot self-attention sub-layer in the decoder stack to prevent positions from attending to subsequent positions.

This masking, combined with fact that the output embeddings are offset by one position, ensures that the predictions for position i can depend only on the known outputs at positions less than i .

▼ 3.2 Attention

attention function: mapping Query and a set of Key-Value pairs to an output

output is computed as a weighted sum of the Values, where the weight assigned to each value is computed by a compatibility function of the Query with the corresponding Key.

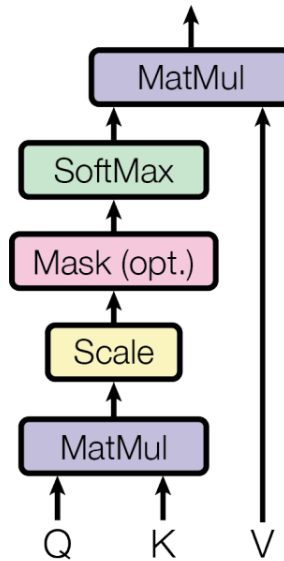
- Query(Q): 현재 처리 중인 단어 벡터.
- Key(K): 각 단어들이 가진 index/이름표 역할. Query와 비교되어 얼마나 관련 있는지 측정하는 데 사용
- Value(V): 각 단어가 담고있는 실제 의미정보. 어텐션 계산 후 최종 거로가값 만들 때 사용

정리하자면

- compatibility function을 통해 Query와 Key 사이의 유사도 계산 (dot product로)
- 계산된 유사도에 softmax함수 적용하여 전체 단어들의 합이 1이 되는 확률 분포 형태의 가중치를 얻는다.
- 이 가중치를 각 Value벡터에 곱한 뒤 모두 더하면(weighted sum) → Query와 가장 관련이 깊은 단어들의 정보가 강조된 최종 결과값을 얻게된다.

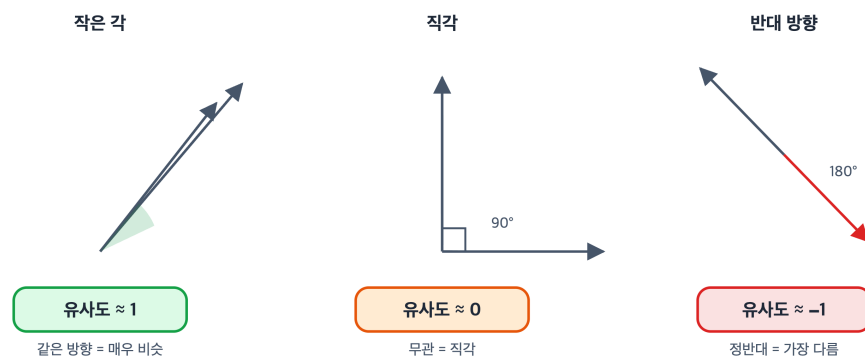
3.2.1 Scaled Dot-Product Attention

Scaled Dot-Product Attention



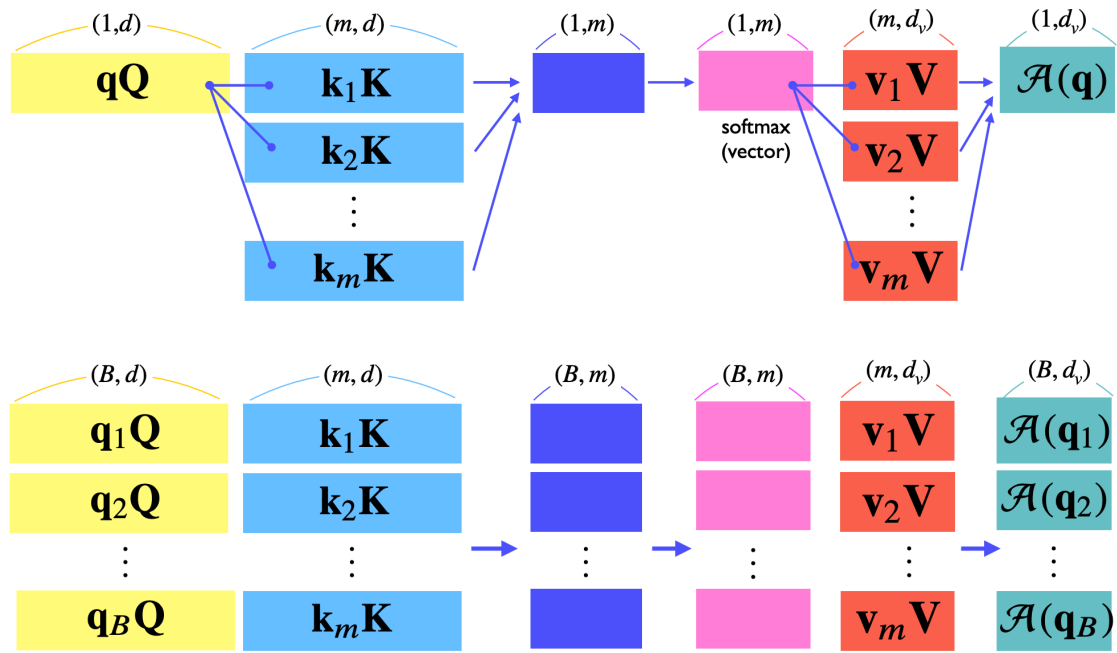
$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

- $\frac{1}{\sqrt{d_k}}$ scaling factor을 곱하는 이유: d_k 가 커질수록 Q, K의 dot product 크기가 매우 커지는 경향. 그리고 softmax함수는 입력값이 커지면 기울기가 0에 가까워지는 saturation구간에 진입하여 학습이 제대로 안이루어짐. 그래서 내적값의 분산을 조절하고, gradient가 살아있는 구간에서 작동하도록 유도하기 위해 scaling factor을 곱한다.
- 현재 처리 중인 단어벡터 Q 와 각 단어가 가진 인덱스/이름표(기준) 역할이되는 K 가 얼마나 비슷한지를 측정하기 위해 코사인 유사도 사용. 두 벡터가 비슷한 정도를 -1~+1사이 값으로 반환.

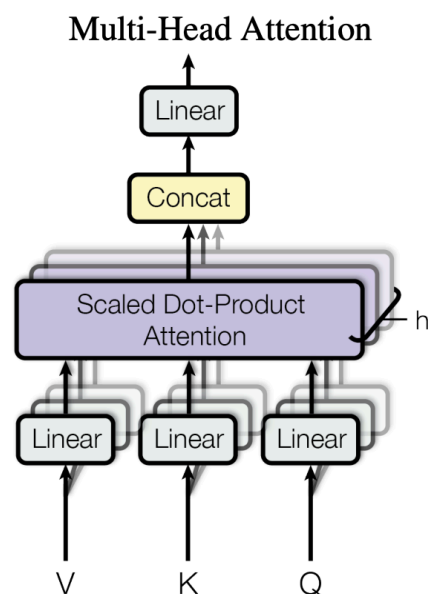


- scaled dot-product값을 확률값으로 처리하기 위해 softmax $\sigma(z_i) = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}$ 활용. 그럼 특정 Q 에 대해 여러가지 K 값들이 갖는 attention값을 확률값으로 만들어주고, 이걸 각 V 에 곱함.

실제론 이걸 행렬로 묶인 set of queries Q 에 대해 동시에 계산함.



3.2.2 Multi-Head Attention

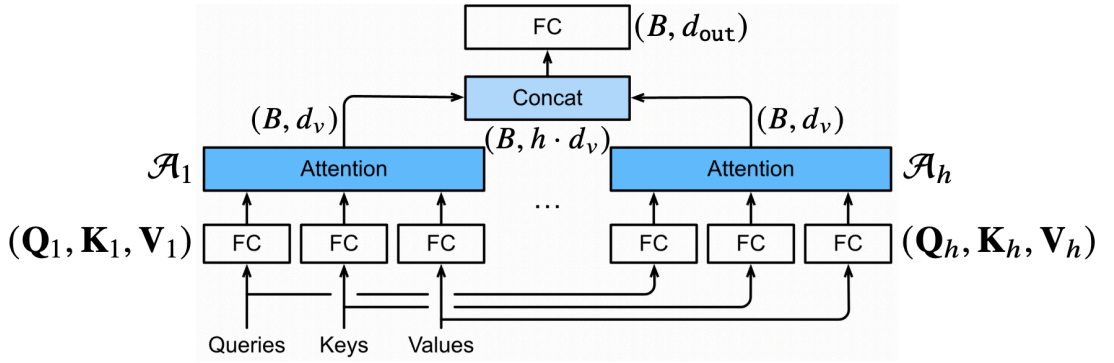


inearly project the queries, keys and values h times with different, learned

linear projections to d_k , d_k and d_v dimensions, respectively. On each of these projected versions of queries, keys and values we then perform the attention function in parallel, yielding d_v -dimensional output values. These are concatenated and once again projected, resulting in the final values

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W^O$$

where $\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$



Where the projections are parameter matrices

$$W_i^Q \in \mathbb{R}^{d_{\text{model}} \times d_k}, \quad W_i^K \in \mathbb{R}^{d_{\text{model}} \times d_k}, \quad W_i^V \in \mathbb{R}^{d_{\text{model}} \times d_v}$$

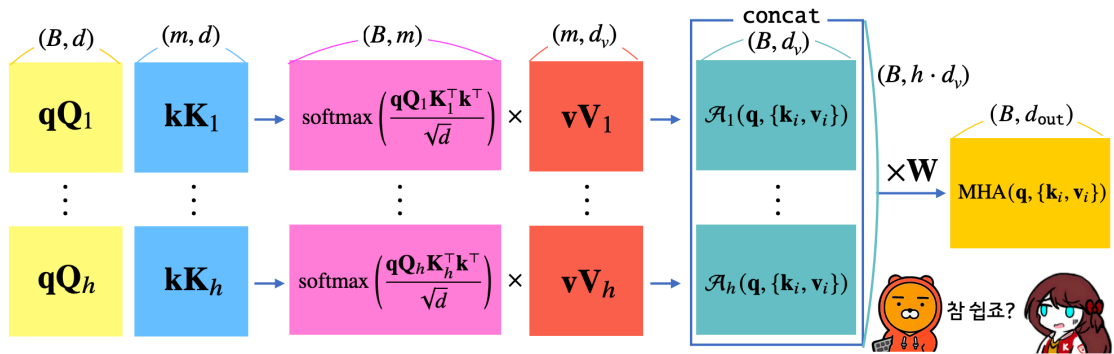
$$W^O \in \mathbb{R}^{hd_v \times d_{\text{model}}}$$

- W_i^Q : 입력 Query를 d_k 차원으로 투영하는 learnable weight matrix
- W_i^K : 입력 Key를 d_k 차원으로 투영하는 learnable weight matrix
- W_i^V : 입력 Value를 d_v 차원으로 투영하는 learnable weight matrix
- W^O : 여러 개의 Head에서 계산된 결과(concat된 벡터)를 최종 출력 차원인 d_{model} 로 다시 투영하는 output weight matrix.

In this work we employ $h = 8$ parallel attention layers(heads)

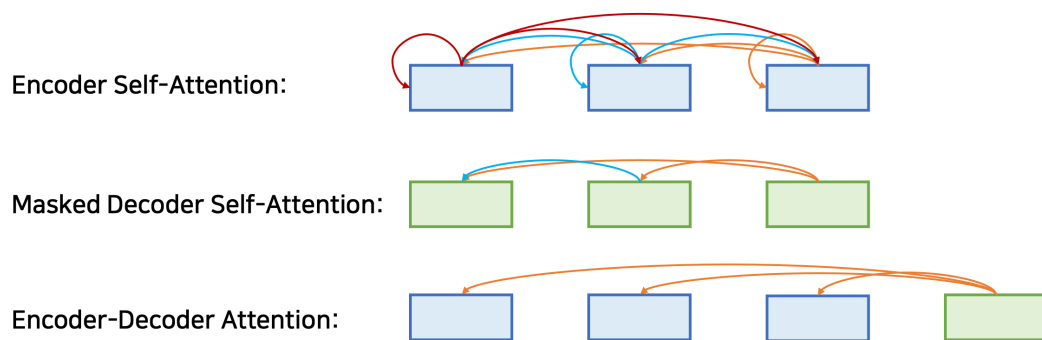
For each of these we use $d_k = d_v = d_{\text{model}}/h = 64$.

Due to the reduced dimension of each head, the total computational cost is similar to that of single-head attention with full dimensionality



- 단일 head로는 모든 정보를 하나의 가중치로 평균내서 계산해야해서 손실/흐려짐 가능성
- 서로 다른 Representation Subspace를 가진 여러 개의 Head를 병렬로 배치
 - 예시) 문법적 역할을 보는 Head, 문맥적 의미를 보는 Head, 대명사 지칭 관계를 보는 Head 등으로 정보를 다각적으로 포착.
 - 여러 Head가 각각 도출한 정보들은 **Concat** 을 통해 합쳐지고 Linear Projection W^0 을 통해 통합된 의미 생성.

3.2.3 Applications of Attention in our Model



- encoder-decoder attention layer:
 - 디코더 각 층에서 Query는 이전 디코더 층 결과값을 사용한다.
 - Key, Value는 인코더의 최종 출력값을 가져와 사용한다.
 - 디코더가 문장 생성할 때, 입력 문장 전체의 정보를 참조할 수 있게 하기 위함. 기존 Seq2Seq모델의 어텐션 메커니즘을 Transformer구조에 맞게 구현.
- encoder의 self-attention layer:

- encoder 각 층에서 모든 K, V, Q는 이전 layer의 output에서 온다.
encoder의 각 position은 바로 이전 encoder layer를 참고할 수 있다.
- 인코더 내의 모든 position이 문장 내의 다른 모든 position을 참조할 수 있도록. 입력 문장의 문맥적 관계 및 의존성 파악.
- decoder의 self-attention layer:
 - decoder의 각 층에서 모든 K, V, Q는 이전 layer의 output에서 온다. 근데 특정 위치 이후의 정보는 참조하지 못하도록 Masking을 적용.
 - 문장을 생성할 때 미래의 정보를 미리 보는 것을 방지하여, 현재 시점까지 생성된 출력값만을 바탕으로 다음 단어를 예측하는 auto-regressive 속성 유지.

▼ 3.3 Position-wise Feed-Forward Network

$$\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2$$

문장 내 모든 위치에 대해 동일한 가중치 (W_1, W_2)를 공유하지만, 인코더와 디코더의 각 레이어마다 다른 파라미터 사용.

각 토큰 위치마다 동일 구조와 파라미터 사용하여 독립적으로 계산. 문장 내의 특정 단어 위치를 하나씩 골라 똑같은 신경망을 통과시키는 방식.

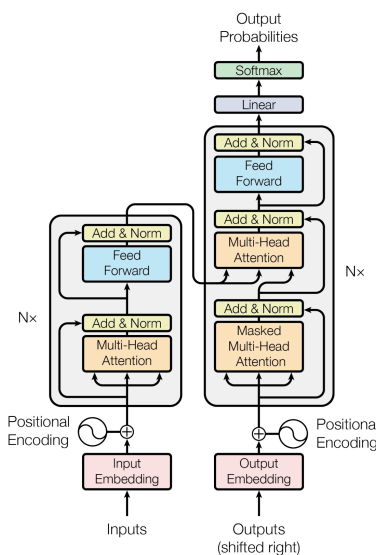


Figure 1: The Transformer - model architecture.

▼ 3.4 Embeddings and Softmax

- Learned Embeddings:
 - 입력을 받아들일 때 token들을 $d_{model} = 512$ 차원의 벡터로 변환. 이때 사용하는 벡터는 학습과정에서 최적화되는 learned embedding 사용. 출력 토큰 생성 시에도 동일하게 적용.
- Linear Transformation & Softmax
 - decoder의 최종 출력은 연속적인 벡터 형태. 다음 토큰이 나올 확률로 바꾸기 위해 두 단계를 거치는데
 - Linear Transformation을 통해 모델의 차원(512)을 vocab size만큼의 차원으로 확장.
 - Softmax적용하여 각 단어가 다음 토큰으로 선택될 확률값을 0~1 사이 값으로 산출.
- Weight Tying
 - 입력 임베딩 층, 출력 임베딩 층, softmax직전 선형 변환 층에서 동일한 weight matrix를 공유
 - 모델 파라미터 수 줄이고 메모리 효율성 높임
 - 임베딩 층에 $\sqrt{d_{model}}$ 을 곱해 뒤이어오는 positional encoding의 크기와 임베딩의 크기를 유사하게 맞추어줌.

▼ 3.5 Positional Encoding

Transformer은 데이터를 순차적으로 처리하지 않고 한 번에 병렬로 처리함 ⇒ 문장 내에서 단어들의 위치 정보를 알 수 없음 ⇒ 시퀀스 순서를 모델이 사용하도록 만들기 위해서 시퀀스에서 토큰들의 상대적/절대적 위치에 대한 정보를 별도로 주입해야 한다 ⇒ **positional encoding**을 input embedding at the bottoms of the encoder&decoder stack에 주입. 임베딩에서 모든 d_{model} 차원을 갖도록 함.

$$PE_{(pos, 2i)} = \sin(pos/10000^{2i/d_{model}})$$
$$PE_{(pos, 2i+1)} = \cos(pos/10000^{2i/d_{model}})$$

▼ <https://yooom.tistory.com/130>

positional encoding 조건:

- positional embedding의 weight가 너무 커져버림에 따른 exploding gradient를 방지하기 위해 -1~1사이의 값으로 표현해야함 (발산하는 함수
↳ closed form이어야함)
- 하나의 값이 하나의 position에 대응되도록 positional embedding의 유일성을 보장해야함. ⇒ closed function인 sin의 반주기 때 원점으로 돌아와 같은 좌표로 돌아오기 때문에, 이를 해결하기 위해 반주기를 엄청 길게 만든다. (분모가 10000인 이유, 충분히 큰 반주기)
- gradient계산을 위해 미분가능한 연속함수여야함.
- position에 따른 position embedding끼리의 거리 차이를 계산할 수 있어야하고, 등간격으로 떨어진 position은 embedding도 그만큼 떨어져있어야함.
 - $PE(x + \Delta x) = PE(x)T(\Delta x)$
 - position의 연속적인 변화량을 반영해주는 선형 변환 행렬T를 찾아내어 Δx 만큼 등간격으로 움직이는 거리(=각도)를 반영하도록.

이걸 전부 만족하는게 위 수식.

▼ 4. Why Self-Attention

1. computational complexity per layer

- Self-Attention layer: 시퀀스 길이 n , 차원 d 에 대해 $O(n^2d)$ 의 복잡도고
- $n < d$ 인 일반적인 번역 환경에서 recurrent network보다 훨씬 빠른 연산 속도

2. amount of computation that can be parallelized

- RNN은 이전 상태 h_{t-1} 가 계산되어야만 현 상태 h_t 를 계산할 수 있는 sequential구조 $\rightarrow O(n)$
- Self-Attention: 모든 위치의 정보를 동시에 처리할 수 있어 병렬화 가능 $\rightarrow$ 필요한 순차적 연산은 $O(1)$ 로 고정.

3. path length between long-range dependencies in the network

- 입력 시퀀스 내 먼 거리에 있는 정보 간의 관계를 학습하는 것이 중요 task
- Self-Attention은 임의의 두 위치 사이 경로를 $O(1)$ 로 유지.
- CNN: $O(\log_k(n))$
- RNN: $O(n)$
- CNN, RNN에 비해 Self-Attention의 path-length가 짧아 먼 거리의 의존성을 훨씬 쉽게 학습할 수 있음.

▼ 5. Training

5.1 Training Data and Batching

- WMT 2014 영어-독일어(450만 문장 쌍) 및 영어-프랑스어(3,600만 문장 쌍) 데이터셋을 사용
- 단어의 의미 단위를 쪼개는 Byte-Pair Encoding 및 Word-piece 토큰화 기법을 사용하여 어휘 사전을 구성
- 배치는 비슷한 길이의 문장들끼리 묶어 처리함으로써 학습 효율을 높임

5.2 Hardware and Schedule

- 8개의 NVIDIA P100 GPU
- 기본 모델(Base model)은 약 12시간, 더 큰 용량의 모델(Big model)은 약 3.5일 동안 train

5.3 Optimizer

- Adam optimizer
- Learning rate 고정하지 않고 초반에 선형적으로 증가(warmup) → 이후엔 train step의 역제곱근에 비례하여 감소시키는 방식

$$lrate = d_{\text{model}}^{-0.5} \cdot \min(step_num^{-0.5}, step_num \cdot warmup_steps^{-1.5})$$

- $d_{\text{model}}^{-0.5}$: 모델의 차원에 따른 scaling factor
- $\min(step_num^{-0.5}, step_num \cdot warmup_steps^{-1.5})$:
 - 학습 초기(warmup_steps까지): learning rate linear increment
 - 그 이후: 학습 진행될수록 learning rate 점진적으로 감소시켜 최적점 근처에서 안정적 수렴하도록

5.4 Regularization

overfitting 방지를 위한 세가지? 전략:

- Residual Dropout — 각 sublayer 출력과 임베딩 합계에 dropout 적용

- Label Smoothing — 모델이 출력값에 지나치게 확신 갖지않도록하여 일반화성능 향상
-

▼ 6. Results

Label Smoothing의 효과:

학습 과정에서 정답을 100% 확신하는 대신 약간의 불확실성을 허용함으로써 모델이 너무 과하게 정답에 치중하는 것을 방지.

이로 인해 Perplexity(모델의 예측 불확실성을 나타내는 지표)는 다소 나빠질 수 있지만, 실제 정답을 맞히는 Accuracy와 기계 번역 품질 지표인 BLEU score는 항상

SOTA 번역 성능:

WMT 2014 영-독(English-to-German) 번역 태스크에서 기존의 SOTA 모델들을 2.0 BLEU 이상 앞서는 28.4 BLEU를 기록

단순히 번역 품질이 높을 뿐만 아니라, 기존 모델들(특히 앙상블 모델들)보다 훨씬 적은 컴퓨팅 자원과 시간으로 더 나은 성능을 달성

연구의 의의와 확장성:

기존의 RNN이나 CNN 기반 모델이 가지고 있던 순차적 계산의 한계를 오직 Attention 메커니즘만으로 극복할 수 있음을 증명

참고할 만한 연구 흐름:

이후 Google's Neural Machine Translation System과 같은 초기 대규모 모델 연구들이 어떻게 병렬화와 효율성 중심으로 변화했는지, 그리고 최근의 거대 언어 모델(LLM)들이 어떻게 이 구조를 변형(예: Masked Language Modeling, Decoder-only 구조 등)하여 사용하고 있는지 살펴보면 좋을듯 (추천 by 켜선생)